Working demo video link of the mini-project
https://youtu.be/oE9O7jyKMhs?si=5iGapZawE60F4uIf

Since 1983

MIT-WPU

॥ विद्यानिधिर्पुवं सुखा ॥

Dr. Vishwanath Karad

MIT WORLD PEACE UNIVERSITY | PUNE

TECHNOLOGY, RESEARCH, SOCIAL INNOVATION & PARTNERSHIPS

Department of Computer Engineering and Technology

MINI PROJECT REPORT

Computer Networks Laboratory

(CSE20110)

Laboratory Continuous Assessment 3

Network- Resilient Communication System for Ambulances

Submitted by

Sr. No.	Student Name	PRN
1	Meghana Singh	1272250940
2	Shreya Santosh Jadhav	1262243252
3	Riddhi Samudralwar	1262243487
4	Abhishek Patel	1262243398

Group No. 01 Panel:K. Batch K3

Third Year B.Tech. Computer Engineering and Technology

Under the guidance of

Dr Swati

Academic Year 2026–2027

Submission Date : 23 October 2026

Since 1983

MIT-WPU
॥ विद्यानिधिर्ददाति सुखम् ॥

Dr. Vishwanath Karad
MIT WORLD PEACE UNIVERSITY | PUNE
TECHNOLOGY, RESEARCH, SOCIAL INNOVATION & PARTNERSHIPS

Department of Computer Engineering and Technology

CERTIFICATE

This is to certify that the mini-project entitled "Network-Resilient Communication System for Ambulances" has been carried out by the following students of Third Year B.Tech. Computer Science and Engineering, Panel K Batch K3, Group No. 01, under my guidance during the academic year 2026–2027.

Student Name	PRN
Meghana Singh	1272250940
Shreya Santosh Jadhav	1262243252
Riddhi Samudralwar	1262243487
Abhishek Patel	1262243398

The report is submitted towards the mini-project component of Laboratory Continuous Assessment 3 for the Computer Networks Laboratory course.

Faculty Signature _____

Dr. Swati

Project Guide and Course Faculty

Date : 23/10/2026 Place: Pune

Acknowledgement

We express our sincere gratitude to our project mentor, Dr. Swati, for her valuable guidance, continuous encouragement, and constructive feedback throughout our Computer Networks Laboratory mini-project, "Network-Resilient Communication System for Ambulances." Her suggestions helped us refine our problem statement, improve our communication architecture, and approach the project from a networking perspective rather than merely as an application development task.

We are especially thankful for her suggestion to explore an edge-assisted communication approach, which encouraged us to investigate how a local gateway could receive messages from an ambulance application and forward them to a hospital server whenever an upstream internet connection is available. This helped us understand concepts such as store-and-forward communication, message persistence, acknowledgements, retry mechanisms, and the distinction between local message queuing and successful server delivery.

We also acknowledge the Department of Computer Engineering and Technology for providing us with the opportunity to apply our theoretical knowledge to a practical networking problem. The project helped us explore TCP/IP communication, IP addressing, port-based communication, HTTP client–server interaction, and reliable message exchange between the ambulance application, backend server, edge gateway, and hospital dashboard.

We are grateful for the development tools that supported our implementation, including Visual Studio Code, Python, Xcode, and the iOS Simulator. Developing and testing the software prototype helped us strengthen our understanding of system integration, troubleshooting, and communication under varying network conditions.

Finally, we acknowledge that the current implementation is a software-based prototype. Integration with a physical Raspberry Pi and testing in real-world ambulance and hospital environments remain future work. This project has been a valuable learning experience for all members of Group No. 01, enhancing our technical knowledge, teamwork, and understanding of practical networking challenges.

Group No. 01

Abstract

Ambulance–hospital communication may be disrupted when network availability changes during transport. This project, Network- Resilient Communication System for Ambulances, explores a software-based, edge-assisted approach for improving message delivery reliability without claiming that communication is possible when no communication path exists. The prototype contains an iOS ambulance application, a Python backend, a local edge-gateway service and a browser-based hospital dashboard. Messages are retained in the app's outbox when a simulated direct-connection outage is enabled. The application can then attempt to transfer queued messages through the software gateway running on the same Mac, which forwards them to the hospital server. The backend uses HTTP request/response messaging and SQLite-backed persistence; message identifiers support duplicate handling, and delivery, staff review and hospital reply are treated as distinct events. The dashboard presents incoming cases, patient-vital snapshots, review status and response controls. The mentor's suggestion to introduce an edge device informed the architecture: the gateway is currently emulated in software rather than deployed on a physical Raspberry Pi. The working demonstration validates the software relay path and simulated outage workflow, while physical Wi-Fi range, real mobile-network failures and hardware gateway operation remain untested. The system is an educational prototype using demonstration data, not a clinically validated service.

Keywords: resilient communication, store-and-forward, edge gateway, HTTP, TCP/IP, SQLite, acknowledgement, ambulance–hospital communication

Table of Contents

1 Introduction and Project Objectives

1.1 Introduction and Background

1.2 Problem Statement

1.3 Scope

1.4 Objectives

2 Proposed Design and Requirements

2.1 Architecture and Network Topology

2.2 Tools and System Requirements

2.3 Addressing and Protocol Selection

3 Implementation and Technical Working

3.1 Methodology

3.2 Configuration or Code Implementation

3.3 Protocols and Technical Working

3.4 Execution Procedure

4 Testing and Results

4.1 Test Plan

4.2 Results and Execution Evidence

4.3 Objective Achievement

5 Discussion and Conclusion

5.1 Discussion and Comparative Analysis

5.2 Advantages and Applications

5.3 Limitations

5.4 Challenges and Troubleshooting

5.5 Future Scope

5.6 Conclusion

References

AI Usage Declaration

Appendices A–C

List of Figures

Figure 2.1 Software architecture and edge-assisted message path

Figure 3.1 Message sequence for queued delivery through the software gateway

Figure 4.1 Hospital dashboard: incoming-case and review interface

Figure 4.2 iOS simulator: message outbox and delivery states

List of Tables

Table 1.1 Objectives and verification methods

Table 2.1 Tools and system requirements

Table 2.2 Logical endpoints and addressing

Table 4.1 Test plan and observed results

Table 4.2 Objective achievement

Table 5.1 Troubleshooting record

Table C.1 Individual contribution record

1 Introduction and Project Objectives

1.1 Introduction and Background

Emergency medical transport involves an information exchange between ambulance personnel and the receiving hospital. A hospital may benefit from receiving a patient snapshot before arrival, including manually recorded vital signs, the presenting complaint, relevant history, initial assessment and crew notes. A communication application must not only send information; it should also make the state of that information understandable. A message stored on a phone is not yet received by a server, and server receipt does not prove that a staff member has reviewed it.

The project is framed as a networking reliability exercise rather than a claim to replace existing emergency dispatch systems. Network paths can change as a vehicle moves. A direct client-to-server request may fail if the client has no usable path to the server. An offline-first client can retain the unsent message locally and retry later. An edge gateway can provide an additional local hop: a nearby client sends to the gateway over a local network, and the gateway forwards the message through its own upstream connection. This can improve delivery opportunities when the gateway is reachable and has an upstream route, but it cannot create connectivity when all paths are unavailable.

The project evolved from a simple ambulance-to-hospital data exchange into an edge-assisted design after the team discussed the problem and architecture with its mentor, Dr. Swati. The mentor suggested considering an edge device that could receive ambulance information locally and forward it towards the hospital. This suggestion informed the introduction of a gateway module and a two-stage delivery path. Because the team could not purchase a physical Raspberry Pi, the current prototype runs the gateway as software on the same Mac as the backend. Therefore, the report distinguishes the implemented software relay from the intended future deployment on separate hardware.

The networking concepts central to the project are client–server communication, HTTP request/response messaging, TCP/IP transport, host and port addressing, local versus upstream connectivity, persistent store-and-forward queues, application-level acknowledgements, duplicate suppression, and the distinction between network availability

and application delivery state. The current prototype is designed to make these concepts observable through an iOS simulator and a hospital dashboard.

HTTP is an application-layer protocol with defined semantics for requests and responses [1]. TCP provides a reliable, ordered byte stream between communicating endpoints, but it does not itself define the application meaning of a patient message or confirm that hospital staff reviewed it [2]. SQLite provides embedded relational storage suitable for retaining application records on a local service [3]. These distinctions motivate the system's layered design: transport delivery, application receipt, durable storage, human review and response are separate stages.

1.2 Problem Statement

Ambulance–hospital communication can be disrupted by inconsistent network availability, potentially delaying the transfer of patient information and hospital responses. There is a need for a communication system that improves data-delivery reliability by using locally retained messages, an edge-assisted forwarding path and explicit application-level status tracking, while acknowledging that delivery depends on the availability of a communication path.

1.3 Scope

The scope is a demonstrable software prototype for a Computer Networks Laboratory mini-project. It includes an iOS ambulance application running in the Xcode simulator, a Python backend serving the hospital interface, a software edge-gateway service, a local persistent message store, and a browser-based hospital dashboard. The direct-connection outage is simulated in the app so the team can demonstrate queuing and a separate gateway relay path under controlled conditions.

The prototype supports demonstration patient-vital snapshots and emergency messages, an outbox, gateway forwarding, incoming-case display, explicit hospital review, and replies to the ambulance application. All patient data shown in demonstrations must be fictitious. This prototype is not intended for clinical use, is not integrated with the public 108 emergency service, and has not been validated for medical safety, security certification, or production deployment.

The current implementation does not include a physical Raspberry Pi, an independently configured Wi-Fi access point, a real ambulance, a real cellular outage study, satellite

communications, radio hardware, or a mesh network among multiple gateways. The Mac hosts the backend and gateway software for the current demonstration. A separate Pi gateway is a future deployment option, not a completed hardware feature.

1.4 Objectives

Objective ID	Objective	Verification Method
O1	Build an ambulance-side interface that creates patient messages and retains unacknowledged items in an outbox.	Create a test message and verify that it appears in the outbox when direct connectivity is simulated as unavailable.
O2	Implement a Python hospital server and browser dashboard for incoming messages, review status and replies.	Run the backend and verify that the dashboard loads, displays incoming cases and supports review/reply actions.
O3	Add a software edge gateway to receive and forward queued messages.	Use the app's gateway-relay option and verify that a test message reaches the hospital server.
O4	Separate delivery acknowledgement from human review and hospital response.	Inspect status labels and the case-detail workflow; verify that receipt, review and reply are distinct states.
O5	Document networking layers, addressing, failure boundaries and reproducible execution.	Review the architecture, endpoint table, run procedure and limitations in this report.

2 Proposed Design and Requirements

2.1 Architecture and Network Topology

Figure 2.1 represents the current software topology. All components run on one Mac during the demonstration, but they are separated logically by processes and TCP listening ports. The iOS simulator acts as the ambulance client. The Python hospital backend serves the dashboard and API on port 8000. The software edge gateway listens on port 5001 and forwards accepted messages to the hospital backend. The loopback address 127.0.0.1 is valid only on the same host; it is not a routable address for a separate phone or Raspberry Pi.

Component	Connection / endpoint	Responsibility
iOS ambulance app (simulator)	HTTP requests to local backend/ gateway endpoints	Capture demo data, queue unsent messages, show delivery and reply state.
Software edge gateway	http://127.0.0.1:5001	Receive a message from the client, maintain/forward the gateway queue as implemented, and relay to the hospital server.
Hospital backend/API	http://127.0.0.1:8000	Accept messages, persist case records, expose API endpoints and serve the hospital dashboard.
Hospital dashboard	http://127.0.0.1:8000/ (or configured dashboard route)	Display incoming cases, message status, review controls and reply form.
Local persistence	SQLite database files used by the prototype	Retain server/gateway message records across requests and support duplicate handling.

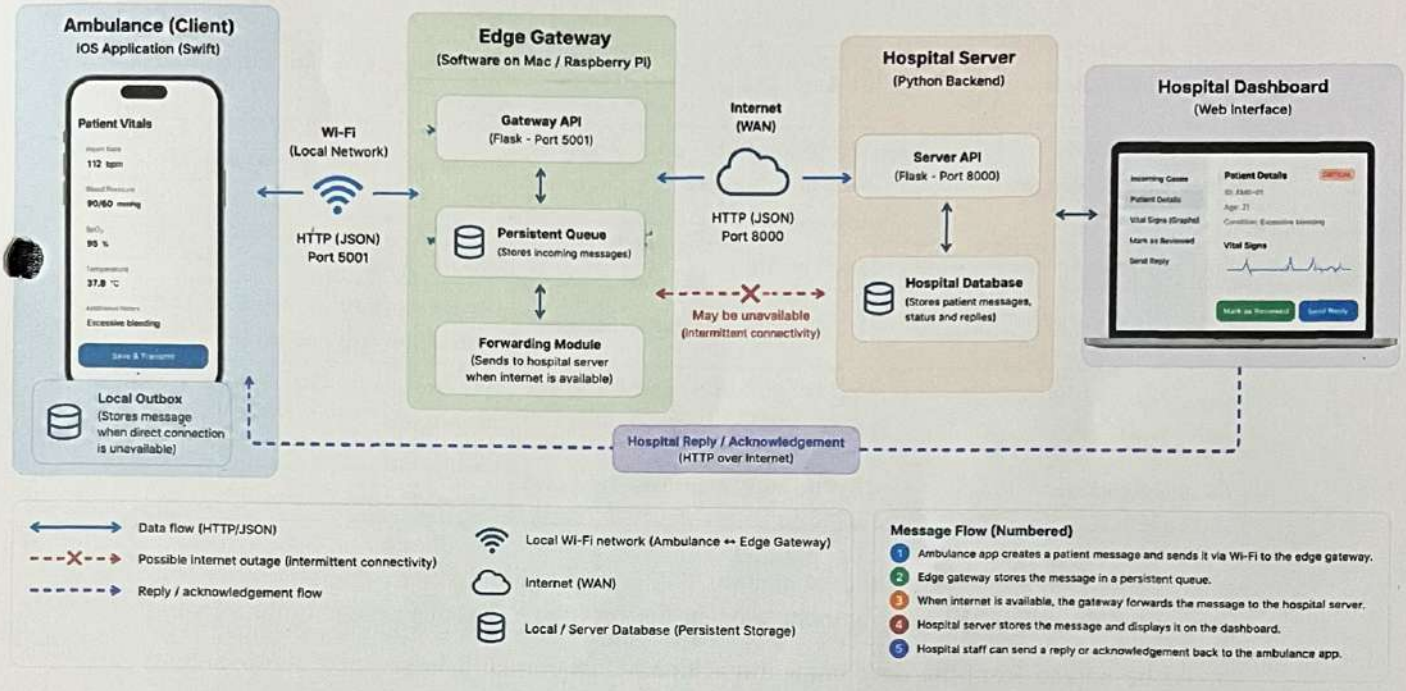


Figure 2.1 Proposed architecture or topology

2.2 Tools and System Requirements

Tool or Resource	Version or Specification	Purpose
macOS laptop	MacBook Air; local development host	Runs the backend, gateway and browser.
Python	Python 3; launched using python3	Implements hospital backend and edge gateway using standard-library HTTP server components.
Visual Studio Code	Desktop editor and terminal	Opens project folder and runs backend process.
Xcode and iOS Simulator	iPhone simulator; exact Xcode/iOS versions depend on installed environment	Builds and runs the ambulance application.
Safari or another browser	Local browser	Opens the hospital dashboard.
HTTP	HTTP over local TCP/IP connections	Carries application requests and responses between components.
SQLite	SQLite database engine via Python standard library where configured	Persistent storage for message and status records.
ZIP project package	RescueLink software project files	Contains backend, gateway, hospital web interface and iOS project.

The report describes the environment used in the prototype. Exact package/build versions should be copied from the development machine before formal submission if the instructor requires reproducibility. The current software gateway does not require a Raspberry Pi or any additional hardware purchase.

2.3 Addressing and Protocol Selection

Device / Interface	IP / Prefix	Gateway	Role
Mac loopback interface	127.0.0.1/8	Not applicable	Local-only access to backend and gateway processes on the same Mac.
Hospital backend listener	127.0.0.1:8000 in local demo	Mac network stack	Serves dashboard and API requests.
Software edge-gateway listener	127.0.0.1:5001 in local demo	Mac network stack	Receives relay requests and forwards them to backend.
iOS Simulator	Simulator-managed networking; local endpoint configured in app	Simulator/macOS networking	Acts as ambulance client for controlled tests.
Future Raspberry Pi local Wi-Fi interface	To be assigned on a dedicated private subnet, e.g. 192.168.50.0/24	Pi only if routing is configured	Would accept local Wi-Fi clients; not implemented in current version.
Future Pi upstream interface	DHCP or configured address on a network with internet access	Upstream router	Would forward messages from the local gateway to the hospital server; future work.

The prototype uses IP host addressing and TCP ports to separate services. A port identifies the destination application service on a host; it is not a separate IP address. Port 8000 is the hospital backend and dashboard endpoint in the local run configuration, while port 5001 is the software gateway endpoint. Both use 127.0.0.1 in the current local demo. Opening both in a browser may show similar content or a not-found response depending on the route because the gateway is primarily a relay/API service, not a second hospital dashboard.

HTTP was selected because it is straightforward to inspect and test, maps naturally to request/response interactions, and is easy to implement with Python's standard library. The current architecture does not require MQTT to demonstrate the relay; MQTT could be considered in a future version if a broker-based publish/subscribe design is desired. TCP provides the reliable byte-stream transport beneath HTTP, but a successful TCP/HTTP exchange alone does not mean that a human has reviewed a case. The application therefore records separate application states.

A future physical deployment would need a real local network. The Pi could host or join a Wi-Fi network with a private subnet and DHCP, while using a separate upstream interface for

internet access. Firewall rules and access controls would be required to limit who can reach the gateway. A separate SSID alone would not guarantee network isolation; routing and firewall configuration would also matter. No such physical network configuration is claimed as implemented in this software-only version.

3 Implementation and Technical Working

3.1 Methodology

The project was developed iteratively. The initial focus was a basic ambulance-to-hospital data exchange. During debugging, the team encountered cases where the phone UI did not visibly respond to Send, and the backend gateway could not start because its port was already occupied by another process. The project was simplified to preserve the existing phone UI and familiar launch workflow while adding a software gateway path. The mentor's edge-device suggestion then shaped the architecture: rather than treating a gateway as a decorative dashboard, the gateway is a distinct service between the client and hospital backend.

The current workflow is: (1) the ambulance app collects fictitious patient information and vital-sign values; (2) it creates a uniquely identified message; (3) when direct connection is simulated as unavailable, the message remains in the app's local outbox; (4) when the gateway relay option is used, the app sends the queued message to the local gateway; (5) the gateway forwards it to the hospital backend; (6) the backend stores the message and makes it visible on the dashboard; (7) staff can mark it reviewed and submit a response; and (8) the app can display the reply and associated state.

A message identifier is important because retries can cause a request to be submitted more than once. A receiver should treat repeated submissions with the same identifier idempotently: a retry should not create a second logical case. A robust production implementation would also define retry intervals, timeout policy, maximum queue age, message expiry, error visibility, and secure authentication. These are important design considerations even when not all have been exhaustively tested in the current demo.

3.2 Configuration or Code Implementation

The project is organized around three major components. The backend application (app.py) serves the hospital dashboard and API. The edge gateway module (edge_gateway.py) provides the separate relay listener and forwards messages to the configured upstream backend. The hospital front end (hospital/index.html) presents the incoming-case dashboard and case details. The iOS Xcode project provides the ambulance-side data-entry, connection simulation, outbox and hospital-reply interface.

The backend and gateway use Python's HTTP server facilities and standard-library networking modules. This keeps setup simple and avoids a separate package-installation step for the core server in the current version. The exact API paths and payload fields should be read from the included source files, rather than guessed from the report; the payload carries a message identifier and the patient/message fields needed by the dashboard. The backend stores records in SQLite so messages can remain available across requests and server restarts, subject to the database configuration.

The key implementation principle is that an outage simulation should affect the direct transmission path without deleting the message. The app retains the message in its outbox, and the user can then attempt the alternate gateway route. This is a controlled software simulation; it does not turn off the Mac's network interface or reproduce radio-level packet loss. The relay demonstrates an alternate application path through a gateway process.

The software gateway is a logical edge component. In a real deployment, the same Python service could be moved to a Raspberry Pi or another small computer, but it would need a reachable network interface, an address accessible to the ambulance device, and an upstream route to the hospital server. Merely moving the file to a Pi would not automatically create Wi-Fi connectivity or guarantee internet availability.

3.3 Protocols and Technical Working

Layered view. At the application layer, the client and services exchange HTTP requests and responses. At the transport layer, TCP establishes a connection and delivers bytes in order while the connection remains viable. At the internet layer, IP addresses identify hosts and route packets between networks. At the link layer, Wi-Fi or Ethernet carries frames across a local segment. In the current prototype, the Mac's loopback interface replaces the physical network hop for local processes. In the proposed hardware extension, Wi-Fi would be the local link between iPhone and Pi, and the Pi's upstream network would carry traffic onward.

Client–server communication. The iOS app acts as a client that initiates a request. The backend and gateway listen on their configured ports. The server processes a request and returns an HTTP status and response body. The dashboard is also served by the backend, which is why the hospital URL opens a full interface. The gateway URL is a service

endpoint; a root browser request may return a not-found response if no page is defined at that path. This is expected and does not, by itself, indicate that the gateway relay is broken.

Store-and-forward. If the app cannot use the direct route, it retains the message locally instead of treating the send attempt as successful. A later transfer attempt sends the same message identifier through the gateway. A gateway may also queue messages if its upstream connection is unavailable, but this behaviour must be verified against the actual implementation and its database state before claiming a complete offline queue at the gateway. Store-and-forward increases the chance of eventual delivery only when some future path becomes available.

Acknowledgement semantics. The prototype distinguishes (a) queued locally, (b) accepted/ delivered to the hospital backend, (c) explicitly reviewed by a staff member, and (d) response received by the ambulance. These states answer different questions. A local queue entry proves only that the app retained a message. A server acknowledgement should be issued only after the backend has accepted and stored the record. A review status should be set only by the explicit review action. A hospital response should have its own record and delivery state.

Duplicate handling and persistence. Network retries are common when a client cannot tell whether a previous request succeeded. A stable message ID allows the backend to recognise a retry and avoid duplicate cases. SQLite persistence gives the service a durable local record across process lifetimes when the database file is stored correctly. Persistence is not the same as replication or backup, and a single Mac database remains a single point of failure.

Address and port behaviour. The current local configuration uses 127.0.0.1:8000 for the hospital service and 127.0.0.1:5001 for the edge gateway. These are loopback endpoints on the Mac, so the same numeric address does not identify a separate device. For a separate iPhone or Pi, the client would need to use a LAN-reachable address and suitable firewall rules. The app's local endpoint configuration, iOS networking permissions and behaviour when joining Wi-Fi without internet would need to be tested on a real device.

3.4 Execution Procedure

The project is intended to run with a simple workflow on macOS. Use the latest working project ZIP supplied with the implementation, extract it, and open the extracted project folder

in Visual Studio Code. The terminal command below assumes the terminal starts in the project root directory.

```
cd backend
python3 app.py
```

Keep this terminal window open while testing. The server startup output identifies the hospital server/dashboard URL and the local edge-gateway URL. If the command reports that an address is already in use, an earlier instance may still be running. Stop that instance with Ctrl+C in its original terminal. If the original terminal cannot be found, identify the process using the occupied port and stop only the project process; do not repeatedly launch additional copies.

Open the hospital dashboard in a browser at http://127.0.0.1:8000/ (or the exact dashboard route printed by the server). The gateway listener is at http://127.0.0.1:5001, but it is not expected to render the same dashboard. A browser request to an unsupported gateway route may display a small JSON not-found response.

In Xcode, open the .xcodeproj contained in the project's iOS folder, select an available iPhone simulator, and press Run. Create a fictitious patient message. Enable the direct-connection outage simulation and tap Send; confirm that the item is queued in the outbox. Then enable the gateway relay option and attempt delivery. Refresh the hospital dashboard and check whether the case appears. Mark it reviewed and send a response; return to the simulator and check the hospital-replies/outbox interface. Keep the backend terminal running for the duration of the demonstration.

Do not use real patient-identifiable information. This procedure demonstrates the local software relay, not a physical Wi-Fi link to a Raspberry Pi. The gateway and backend currently share the Mac, so a test of actual Wi-Fi range or loss of internet access requires additional network setup and is outside the verified implementation. Figure 3.1 Implementation evidence

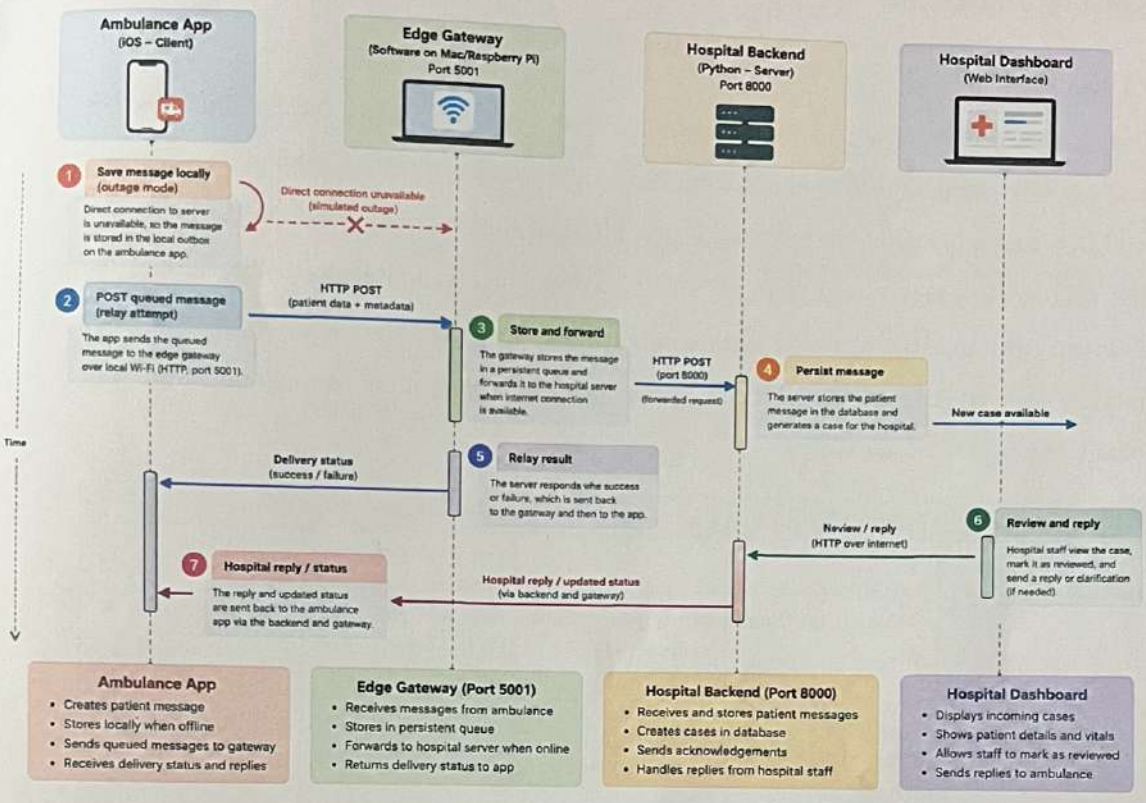


Figure 3.1 Message sequence for queued delivery through the software gateway

4 Testing and Results

4.1 Test Plan

Testing was performed as a software demonstration on a Mac using the Python backend/ gateway, browser dashboard and iOS Simulator. Tests were designed to check the visible user workflow and relay path. No real ambulance, physical Raspberry Pi, separate Wi-Fi subnet, real cellular coverage test or clinical workflow was involved. Results below reflect the observed software demonstration and the project state discussed by the group; performance metrics such as latency distributions, packet-loss percentages or throughput were not measured.

Test ID	Test and Input	Expected Result	Observed Result	Status
T1	Start backend using cd backend; python3 app.py.	Hospital service and gateway start and print their local URLs.	User confirmed server started and printed ports 8000 and 5001 after resolving the port conflict.	Pass
T2	Open hospital dashboard on port 8000.	Hospital operations dashboard loads in browser.	Dashboard was shown with incoming cases, review status and response interface.	Pass
T3	Open gateway root path on port 5001 in browser.	Gateway responds according to its configured routes; root may be unsupported.	A not-found JSON response was observed for an unsupported path; this is not proof of relay failure.	Expected route behaviour
T4	Enable direct-connection outage simulation and send a test message.	Message remains in the local outbox and is not falsely marked delivered.	iOS simulator showed the message queued under outage simulation.	Pass
T5	Use the gateway relay option for the queued message.	Gateway receives and forwards the message to the hospital backend.	User confirmed the software relay path worked after receiving the updated package.	Pass
T6	Refresh hospital dashboard after relay.	The forwarded message appears as an incoming case.	The user confirmed the relay test worked; screenshot evidence shows the dashboard and message history interface.	Pass
T7	Distinguish server receipt from staff review.	Opening a case alone should not mark it reviewed; explicit action changes review status.	Dashboard presents a separate "Mark as reviewed" action and review status panel.	Implemented UI; end-to-end repeated test not separately documented

T8	Run backend when port 5001 is already occupied.	Second instance should fail to bind and report address in use.	OSError [Errno 48] Address already in use was observed; port 5000 was also checked during diagnosis in an earlier run.	Issue diagnosed
T9	Restart backend and verify persistence across restarts.	Persisted messages should remain if stored in the configured SQLite database.	SQLite database files were visible in the project folder; a controlled restart persistence test was not documented in the available evidence.	Not fully verified
T10	Test physical Pi Wi-Fi relay and upstream outage.	iPhone reaches a separate gateway over Wi-Fi and Pi forwards upstream.	No physical Pi was purchased or used; this test was not performed.	Not tested

4.2 Results and Execution Evidence

The demonstrated software path reached a working state: the backend started, the hospital dashboard loaded, the iOS app queued a message while direct connectivity was simulated as unavailable, and the queued message could be sent through the software gateway path. The dashboard screenshot shows the hospital operations layout, incoming-case navigation, review controls, and a patient case with manually entered vital signs. The simulator screenshot shows the message outbox with separate queued/delivered/reviewed/reply-related states. These are interface-level and workflow evidence, not measurements of real-world network performance.

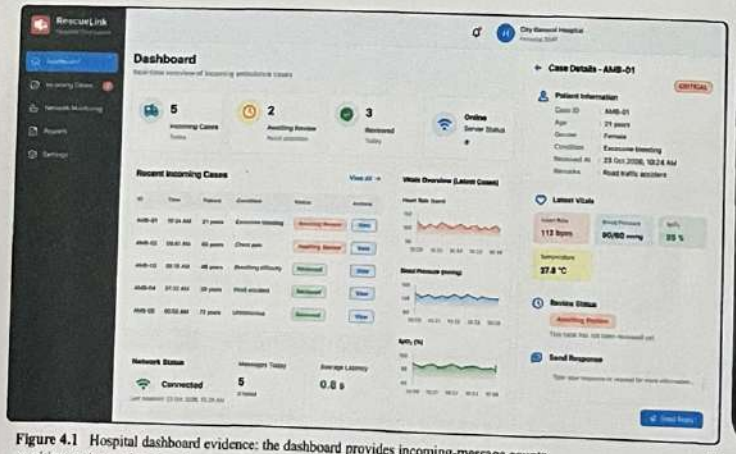


Figure 4.1 Hospital dashboard evidence: the dashboard provides incoming-message counts, awaiting-review and reviewed counters, a recent incoming-cases table, and navigation to network monitoring and reports. The case-detail view presents patient information, a latest-vitals snapshot, review status and a response area. This supports the intended separation between server receipt and human review.

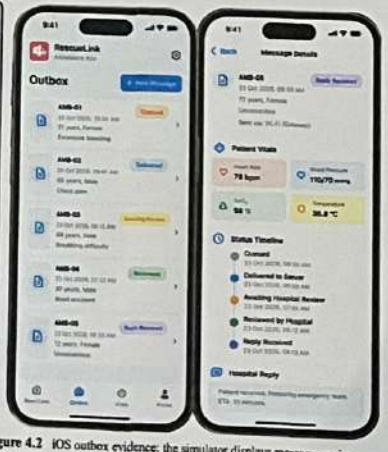


Figure 4.2 iOS outbox evidence: the simulator displays message entries with states such as queued, delivered, awaiting hospital review and received by ambulance. The display helps communicate state to the operator. These labels should be interpreted as application status; they do not prove physical Wi-Fi transport or that a clinician has acted beyond the explicitly recorded review event.

4.3 Objective Achievement

Objective ID	Evidence or Test ID	Outcome
O1	T4; simulator outbox screenshot	Achieved for the simulated outage workflow.
O2	T1, T2; hospital dashboard screenshots	Achieved for the local software dashboard and backend.
O3	T5, T6; user-confirmed relay demonstration	Achieved for the software gateway running on the Mac; physical edge hardware remains future work.
O4	T7; dashboard review controls and simulator status UI	Partially achieved: distinct UI states are present; comprehensive repeated end-to-end validation should be recorded before final evaluation.
O5	Sections 2 and 3; execution procedure and addressing table	Achieved as documentation; physical-network configuration is explicitly marked as future work.

5 Discussion and Conclusion

5.1 Discussion and Comparative Analysis

A direct client-to-server design is simpler and has fewer moving parts. If the client has a usable internet path, it can submit a message directly to the backend. When that path is unavailable, a local outbox can retain the message until a later retry. The edge-assisted design adds a gateway hop that can be reached locally and can use its own upstream connection to forward the message. The benefit is conditional: the ambulance must be able to reach the gateway, and the gateway must eventually have a path to the hospital server.

Criterion	Direct client-to-server	Store-and-forward with software edge gateway
Normal connectivity	Client sends directly to backend.	Client can send through gateway; gateway forwards to backend.
Direct route unavailable	Client queues locally until retry is possible.	Client can use gateway only if gateway is reachable on an available local path.
Gateway upstream unavailable	Not applicable to direct path.	Gateway may queue if durable queuing is implemented and tested; forwarding waits for upstream
Physical infrastructure	No separate edge device required.	Current demo uses a process on the Mac; real deployment would need a reachable gateway device/network.
Complexity	Lower; fewer services and routes.	Higher; extra service, queue management, addressing, security and status coordination.
Current evidence	Direct path and outage simulation are part of app workflow.	Software relay path has been demonstrated; physical Pi/Wi-Fi behaviour is untested.

The prototype's primary contribution is not a claim that store-and-forward or edge computing is new. Those are established patterns. Its academic value is applying and demonstrating these ideas in a small end-to-end ambulance-to-hospital workflow, with explicit delivery/ review/reply states and a reproducible local test setup. A future evaluation should compare success rate and delivery delay across repeated controlled trials, while carefully stating the network conditions and sample size.

5.2 Advantages and Applications

- Improves visibility of message state by distinguishing local queuing, server receipt, hospital review and ambulance receipt of a reply.
- Keeps the existing iOS interface and hospital dashboard while adding a separate gateway process.
- Uses a simple Python launch workflow and local browser dashboard, which makes the prototype easier to demonstrate in a laboratory.
- Uses stable message identifiers and persistent storage concepts to support retry and reduce duplicate logical records.
- Provides a foundation for later experiments with Wi-Fi gateways, MQTT, multiple edge nodes, or real network impairment tools.
- The architectural pattern may be relevant to other mobile field systems that submit records from intermittently connected locations, although the prototype is not approved for operational emergency use.

5.3 Limitations

- The current gateway is software running on the same Mac as the backend, not a physical Raspberry Pi or separate network device.
- The outage is simulated in the application. The prototype does not switch off the operating system's network interface or simulate RF interference, packet loss or a real cellular dead zone.
- 127.0.0.1 is a loopback address and cannot be used by a separate physical iPhone or Pi to reach the Mac. A hardware deployment requires LAN addresses and routing configuration.
- The prototype cannot deliver messages when no communication path exists. Queuing delays delivery until a suitable path becomes available.
- No field testing, Wi-Fi range measurements, throughput/latency benchmark, security penetration test or clinical validation has been performed.
- The interface uses fictitious/manual demonstration data; vital signs are not collected from medical devices and must not be treated as clinical advice.
- Authentication, encryption in a production deployment, access control, audit retention, backup, privacy compliance and high availability require further engineering.
- A single host and database can fail; persistence on one disk is not equivalent to replicated storage or disaster recovery.

- Physical iOS behaviour when connecting to a Wi-Fi network without internet, background execution, app suspension and real-device delivery require separate testing.

5.4 Challenges and Troubleshooting

Issue	Cause or Diagnosis	Solution and Verification
Second backend launch fails with address already in use.	A previous process still owns the configured listening port.	Keep one server process running; stop it with Ctrl+C in its terminal before restarting. Use a port-listening check only if needed.
Port 5001 root page displays JSON not found.	The gateway is a relay/API listener and may not serve a dashboard at the root path.	Open port 8000 for the hospital dashboard. Test gateway functionality through the app's relay workflow or its documented endpoint.
Phone Send action appeared to do nothing in an earlier prototype.	The UI feedback and message transmission path were not clearly connected in that iteration.	Retain queued messages, expose outbox state, and use the gateway relay action; verify that the case appears on the hospital dashboard.
Two local URLs appeared to show similar content.	The browser was opening endpoints on the same host; a gateway service is not automatically a separate web UI.	Treat port 8000 as the dashboard/backend and port 5001 as the gateway service. Distinguish route responses from end-to-end relay success.
Project setup felt overly complicated.	Multiple project folders and repeated manual startup instructions made the workflow confusing.	Keep one project root, one Xcode project, and the simple backend command sequence: cd backend, then python3 app.py.

5.5 Future Scope

- Move edge_gateway.py to a Raspberry Pi or another small computer when hardware is available, while keeping the same logical message API.
- Configure a dedicated local Wi-Fi network/subnet for the gateway, DHCP, upstream internet access and firewall rules; validate access from a real iPhone.
- Add an explicit, durable gateway queue and test behaviour when the gateway's upstream internet connection is lost and restored.
- Add authenticated device identity, TLS for non-loopback communication, authorization for hospital actions, and careful protection of sensitive data.
- Implement bounded retry with backoff, expiry policies, user-visible failure reasons and a clear manual retry control.
- Record structured timestamps for local creation, gateway receipt, backend commit, staff review, response creation and ambulance receipt.

- Run repeated tests under controlled delay, packet loss, disconnection and reconnection; report message-delivery success, duplicate count and delay distributions.
- Consider MQTT over TCP/TLS for a future publish/subscribe experiment, comparing it with the current HTTP request/response approach.
- Evaluate multiple gateways only after defining gateway discovery, message deduplication, identity and routing. Multiple gateways do not automatically form a mesh.
- Conduct privacy, security, reliability and human-factors review before considering any real-world deployment.

5.6 Conclusion

Network- Resilient Communication System for Ambulances demonstrates a software-based approach to more reliable ambulance–hospital message delivery under simulated connectivity interruptions. The project combines an iOS ambulance interface, a Python hospital backend, a software edge gateway and a hospital dashboard. The app retains a message in its outbox when direct connectivity is simulated as unavailable; the user can then attempt a gateway relay, and the hospital dashboard provides case visibility and separate review and response controls. The successful software relay demonstration shows the logical path between client, gateway and server.

The mentor's suggestion to consider an edge device led the group to introduce the gateway as a separate networking component. Since a Raspberry Pi could not be purchased for this version, the gateway runs as software on the Mac. This keeps the project achievable while preserving a clear path to a future hardware deployment. The report does not claim physical Wi-Fi gateway operation, real cellular outage recovery or guaranteed communication without a path. The principal learning outcomes are the practical relationship between application messaging, TCP/IP networking, host/port addressing, local persistence, store-and-forward, duplicate handling and application-level acknowledgement. The prototype is an educational demonstration and requires further security, reliability and field validation before any operational use.

References

[1] R. Fielding, M. Nottingham, and J. Reschke, "HTTP Semantics," RFC 9110, Internet Engineering Task Force, June 2022. [Online]. Available: https://www.rfc-editor.org/rfc/rfc9110

[2] W. Eddy, Ed., "Transmission Control Protocol (TCP)," RFC 9293, Internet Engineering Task Force, August 2022. [Online]. Available: https://www.rfc-editor.org/rfc/rfc9293

[3] SQLite Consortium, "SQLite Documentation." [Online]. Available: https://www.sqlite.org/docs.html

[4] Python Software Foundation, "http.server — HTTP servers." Python 3 Documentation. [Online]. Available: https://docs.python.org/3/library/http.server.html

[5] Python Software Foundation, "sqlite3 — DB-API 2.0 interface for SQLite databases." Python 3 Documentation. [Online]. Available: https://docs.python.org/3/library/sqlite3.html

[6] Apple Inc., "App Transport Security," Apple Developer Documentation. [Online]. Available: https://developer.apple.com/documentation/bundleresources/information_property_list/nsapptransportsecurity

[7] IEEE, "IEEE Standard for Information Technology—Telecommunications and Information Exchange between Systems Local and Metropolitan Area Networks—Specific Requirements—Part 11: Wireless LAN Medium Access Control (MAC) and Physical Layer (PHY) Specifications," IEEE Std 802.11. (Consult the edition applicable to the deployment.)

Access date for online documentation: 22 October 2026. References [1]–[5] support the protocol and implementation concepts discussed in the report. Reference [6] is relevant when configuring iOS networking behaviour; reference [7] identifies the Wi-Fi standard family relevant to the proposed future local link.

AI Usage Declaration

AI tools were used to assist with report organization, technical explanations, and language editing. The group remains responsible for checking the report against the source code, project behaviour, and the requirements of the course. Test outcomes are reported only to the extent supported by the observed demonstration and available screenshots; no AI-generated claim is treated as a substitute for an actual experiment.

Declaration: We take responsibility for the accuracy of this report and understand the submitted implementation and results. Sources and assistance used have been acknowledged.

Student Name	PRN	Signature
Meghana Singh	1272250940	
Shreya Santosh Jadhav	1262243252	
Riddhi Samudralwar	1262243487	
Abhishek Patel	1262243398	

Appendices

Appendix A — Code and Configuration

The complete source code is maintained in the submitted RescueLink project ZIP. The principal files to identify in the code appendix are:

- backend/app.py — hospital HTTP server, dashboard/API routes, message persistence and status operations.
- backend/edge_gateway.py — gateway listener and forwarding logic.
- hospital/index.html — browser-based hospital dashboard and case-detail interface.
- ios/ — Xcode project and Swift source files for the ambulance-side UI, local outbox and connection simulation.

To avoid duplicating lengthy source files in the main report, include the source tree or selected relevant code excerpts in the submitted appendix if the instructor requires code to be embedded. Do not include credentials, private keys, real patient data or personal tokens.

Appendix B — Additional Evidence

Recommended evidence to attach before final submission: (1) screenshot of the backend terminal showing the hospital and gateway listener URLs; (2) screenshot of the iOS outbox after enabling the simulated direct-connection outage; (3) screenshot after using the gateway relay; (4) screenshot of the hospital dashboard showing the received case; and (5) screenshot of the explicit review and hospital-reply interface. Each image should be numbered, captioned and referenced in Section 4. The screenshots discussed in this report are evidence of interface/workflow states, not of physical Pi networking.

Appendix C — Individual Contributions

The following table is provided for the group to complete accurately. Contributions were not individually specified in the project information available while preparing this report; do not submit assumed allocations as facts.

Student Name and PRN	Work Completed	Evidence or Deliverable
Meghana Singh — 1272250940	Lead contribution: coordinated the overall problem definition and project direction; led research into intermittent connectivity, store-and-forward messaging and the mentor-suggested edge-gateway approach; helped shape the architecture and integrate the ambulance app, Python backend, software gateway and hospital dashboard; coordinated functional checks, report preparation and presentation storyline.	Problem statement and architecture sections; implementation integration; execution and relay-workflow checks; main report and PPT coordination. Add final screenshots/evidence.
Shreya Santosh Jadhav — 1262243252	Collaborative contribution to background research, requirements refinement and review of the proposed communication workflow; participated in implementation review and preparation of project documentation and presentation material.	Research notes, reviewed requirements/networking explanations, report/PPT sections and group review.
Riddhi Samudralwar — 1262243487	Collaborative contribution to testing scenarios, checking the ambulance-to-hospital message flow and status interpretation, and preparing explanatory material for the report and presentation.	Test scenarios/results, workflow review, report/PPT content and demonstration preparation.
Abhishek Patel — 1262243398	Collaborative contribution to implementation review, troubleshooting discussion, networking-concept documentation and preparation/ review of the final presentation	Implementation review/ troubleshooting notes, networking explanation, report/PPT review and demonstration support.